

# Bounding Wireframes for Implicit Surfaces: Cube and Sphere Outlines via Selective Edge Overlay

engvis

June 19, 2026

## 1 Problem Statement

The `engvis` viewer renders implicit surfaces  $\{f = 0\}$  restricted to a working domain  $\Omega$ :

- `gyroid`:  $\Omega = [-1, 1]^3$  (cube)
- `gyroid-sphere`:  $\Omega = \{\mathbf{p} \in \mathbb{R}^3 : \|\mathbf{p}\| \leq R\}$  (ball,  $R = 1$ )

When the surface is open (its boundary lies on  $\partial\Omega$ ), the viewer needs a visual cue showing *which* domain the surface lives in. Without this cue, the eye cannot tell whether a curved silhouette is a feature of  $f$  or merely the clip against  $\partial\Omega$ .

We add a wireframe outline of  $\partial\Omega$  rendered as line segments, so  $\Omega$  is unambiguous while the lit surface itself stays free of triangle-edge clutter.

## 2 Design Choices

### 2.1 Why selective edge overlay, not a separate line shader

`engvis` already has a triangle-line pipeline in `OverlayRenderer` that turns each mesh edge  $(A, B)$  into a quad of constant pixel width. Hooking the bounding outline into this pipeline avoids introducing a parallel line shader and reuses the MSAA / depth state set up for the rest of the frame.

The edge overlay, however, was written to draw *every* triangle edge of *every* mesh. Drawing every edge of the gyroid surface ( $\sim 10^4$  triangles) would bury the bounding outline. We therefore introduce a per-node opt-in flag, so only the wireframe mesh contributes to the edge pass:

```
pub struct SceneNode {  
    // ...  
    /// Whether to draw this node's edges in the edge-overlay pass.  
    pub render_edges: bool,  
}
```

In the renderer:

```
let skip_mesh = matches!(mode, OverlayDrawMode::Edges)  
                && !node.render_edges;  
if !skip_mesh && let Some(mesh_idx) = node.mesh_index { ... }
```

## 2.2 Why degenerate triangles, not a dedicated line mesh

The mesh data structure `Mesh` stores triangles in `indices: Vec<u32>`. Adding a separate “line list” channel would force every downstream pass (vertex extraction, AABB, GPU upload) to carry two parallel buffers.

We instead encode each line segment  $(A, B)$  as a *degenerate triangle*  $(A, B, A)$ . This works because:

- `extract_edge_indices()` unpacks each triangle  $(i_0, i_1, i_2)$  into edges  $(i_0, i_1), (i_1, i_2), (i_2, i_0)$ , which for  $(A, B, A)$  becomes  $(A, B), (B, A), (A, A)$ . The first two are the same line in opposite directions; the third has zero length and contributes a zero-area quad.
- In the PBR rasterisation pass the triangle has zero area, so no fragments are produced — the wireframe never appears as a solid surface.

## 3 Geometry

### 3.1 Cube wireframe (gyroid)

The cube  $[-1, 1]^3$  has 8 corners  $V_i$  and 12 edges  $E_k = (V_{a_k}, V_{b_k})$ . Each edge  $E_k$  becomes a degenerate triangle in the mesh:

$$\text{indices} += [2k, 2k + 1, 2k].$$

### 3.2 Sphere wireframe (gyroid-sphere)

The sphere of radius  $R = 1$  is approximated by a latitude–longitude grid. With  $n_{\text{lat}} = 12$ ,  $n_{\text{lon}} = 24$  we generate

- $n_{\text{lon}}$  *meridians*, each subdivided into  $2n_{\text{lat}}$  polar arcs:

$$\mathbf{p}(\varphi, \theta) = R (\sin \theta \cos \varphi, \cos \theta, \sin \theta \sin \varphi), \quad \theta \in [0, \pi], \varphi = \frac{2\pi i}{n_{\text{lon}}}.$$

- $n_{\text{lat}} - 1$  *parallels*, each subdivided into  $2n_{\text{lat}}$  azimuthal arcs:

$$\mathbf{p}(\varphi) = (R \sin \theta_j \cos \varphi, R \cos \theta_j, R \sin \theta_j \sin \varphi), \quad \theta_j = \frac{\pi j}{n_{\text{lat}}}.$$

Total segment count:  $n_{\text{lon}} \cdot 2n_{\text{lat}} + (n_{\text{lat}} - 1) \cdot 2n_{\text{lat}} = 2n_{\text{lat}}(n_{\text{lon}} + n_{\text{lat}} - 1)$ , which for  $(12, 24)$  gives 840 segments.

## 4 Pipeline

## 5 Comparison

## 6 Implementation Summary

- `src/main.rs`
  - `build_box_wireframe()` — 12-edge cube outline.
  - `build_sphere_wireframe(r, n_lat, n_lon)` — lat/lon sphere outline.

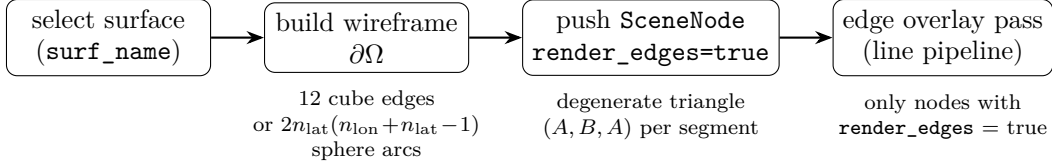


Figure 1: Wireframe construction and selective edge-overlay pass. The PBR pass ignores the wireframe mesh because every triangle has zero area; the edge pass ignores every other node because they have `render_edges = false`.

| Approach                                     | Reuses overlay pipeline? | Per-node control? |
|----------------------------------------------|--------------------------|-------------------|
| New line shader                              | no                       | n/a               |
| Edge overlay, all meshes                     | yes                      | no                |
| Edge overlay, <code>render_edges</code> flag | yes                      | <b>yes</b>        |

Table 1: Trade-offs of bounding-outline rendering strategies. The chosen approach is the only one that reuses the existing pipeline *and* keeps the surface mesh’s triangle edges hidden.

- In `App::build_scene()`: choose cube / sphere wireframe by `surf_name`, push as an extra node with `render_edges = true`.
- In `App::ui()`: enable `frame.render_state.edge_opts` (colour, line width).
- `clip_mesh_to_ball(..., r=1.0)`: clipping ball radius now matches the wireframe.
- `crates/engvis-core/src/scene.rs`: add `render_edges: bool` field to `SceneNode` (default `false` in `Scene::single_mesh`).
- `crates/engvis-renderer/src/renderer.rs`: in `render_overlay_node`, skip nodes with `render_edges = false` when `mode = Edges`.
- `crates/engvis-renderer/src/gltf_loader.rs`: initialise `render_edges = false` for imported nodes.

## 7 Result

- **gyroid**: a thin cube outline  $[-1, 1]^3$  frames the lit gyroid mesh; the surface itself shows no triangle edges.
- **gyroid-sphere**: the lat/lon sphere ( $R = 1$ ) frames the spherically-clipped gyroid patch. The surface boundary lies exactly on this sphere because `clip_mesh_to_ball` cuts at the same radius.
- Both outlines respect MSAA, depth, and the configurable line width / colour from `EdgeRenderOptions`, with no extra shader code.